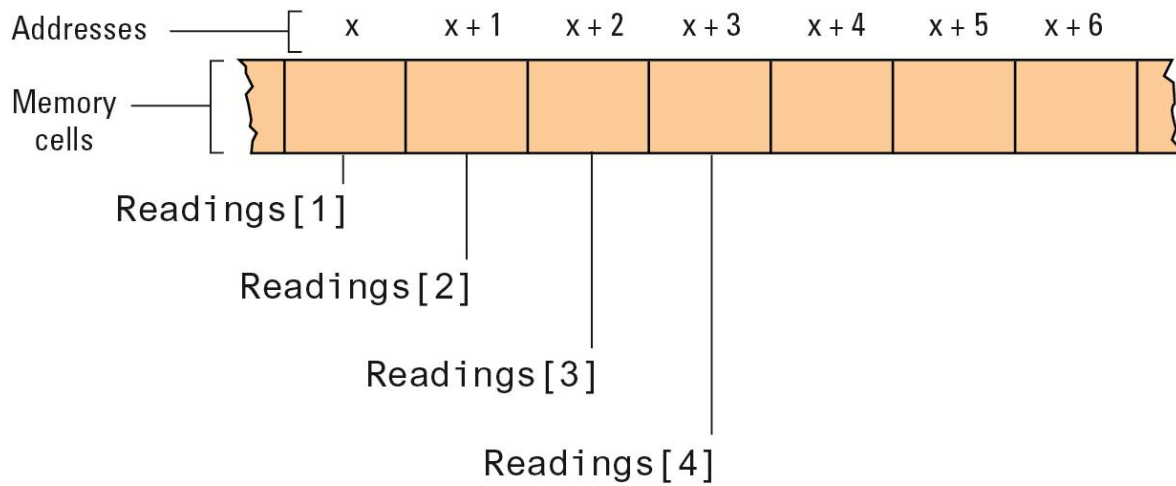


Ch8 Data Abstractions

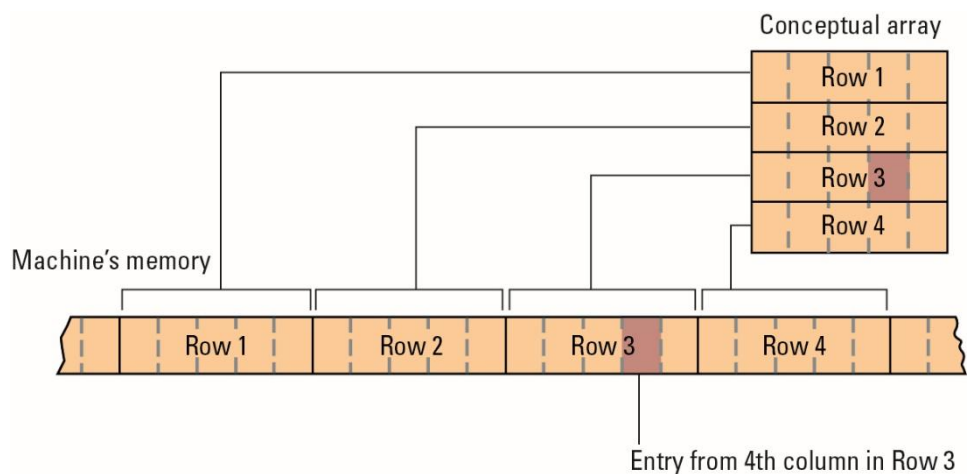
Array 陣列

- 一連串一樣資料型態的資料
- 透過 Indices(指數)表示位置
- 二維陣列(two dimensional array)有行跟列兩個索引值

實際儲存 Array



實際儲存 二維陣列



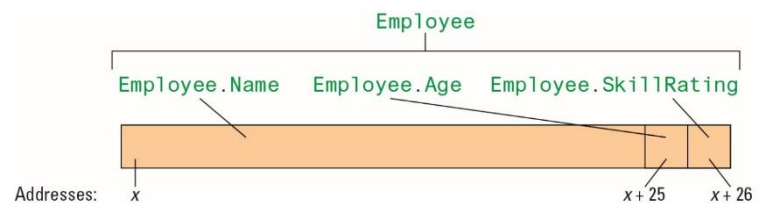
Aggregates (Ex: Struct)

- 一串不同資料型態或大小的資料
- 每項資料稱作 Field(欄位)
- 通常用名稱取值

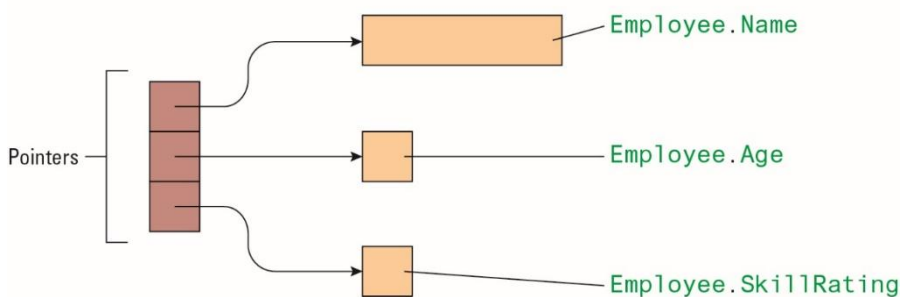
實際儲存 Aggregate

- 儲存在連續的 Memory Block

可以計算其中 Field 的儲存位址



- 存在不同的位址，以指標連結



b. Aggregate fields stored in separate locations

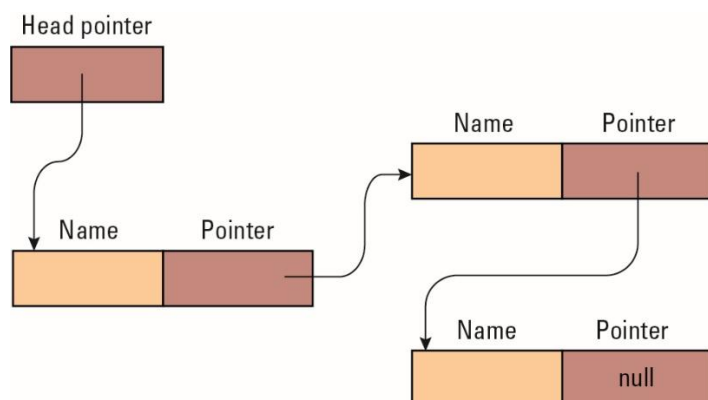
List

- 一串資料序列
- Head：List 的第一個項目
- Tail：List 除了 Head 之外的其他項 // 課本敘述有問題，說是最後一項

Linked List

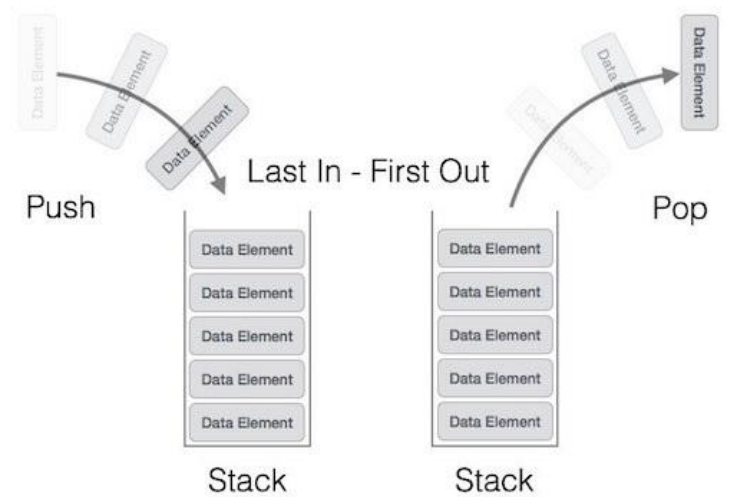
每一項都帶有一個 Pointer，指向下一項的 Address

- Head Pointer：記錄第一項的 Address
- NULL：空指標，是一個特殊常數，不指向任何有效記憶體指標值，代表沒有東西
- 最後一項的 Pointer 為 NULL Pointer (a.k.a NIL Pointer)



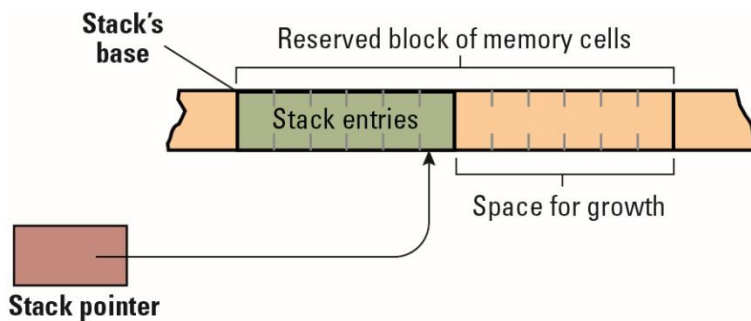
Stack 堆疊：就是疊高疊起來啦

- LIFO (Last in first out)
- Top：stack 的最頂端
- Bottom/Base：stack 最底下，放最早放入的元素
- Pop：移除 stack 最頂端的元素
- Push：把元素放入 stack 最頂端



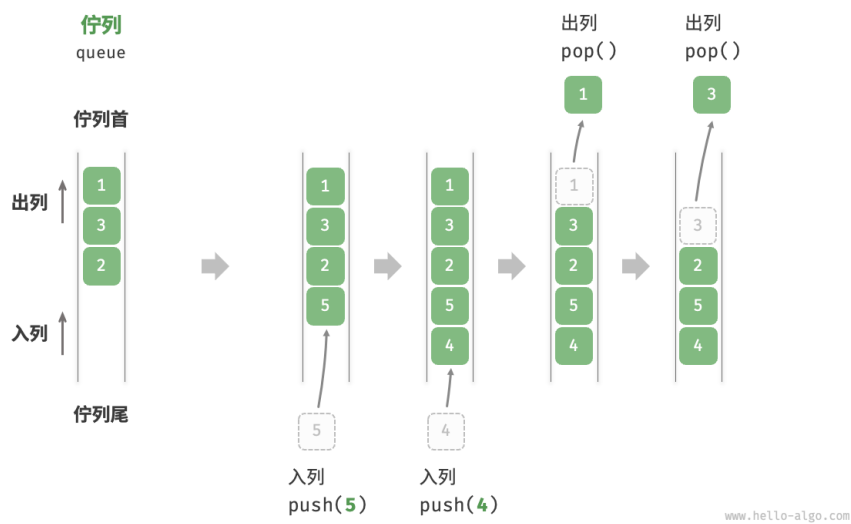
實作 Stack

- Stack 通常用連續記憶體 (Array) 儲存
- Stack Pointer 指向 Stack 的 Top



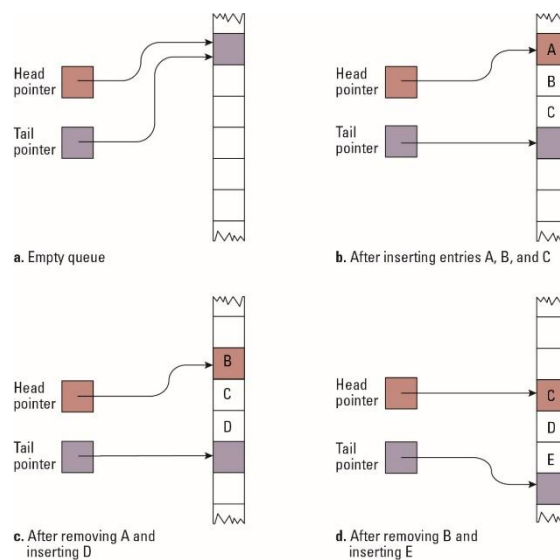
Queue 佇列：就是排隊啦

- FIFO(First in first out)
- Head：隊伍最前面
- Tail：隊伍尾巴
- Pop：移除 Head 的元素 排到了
- Push：把元素放入 Tail 加入排隊



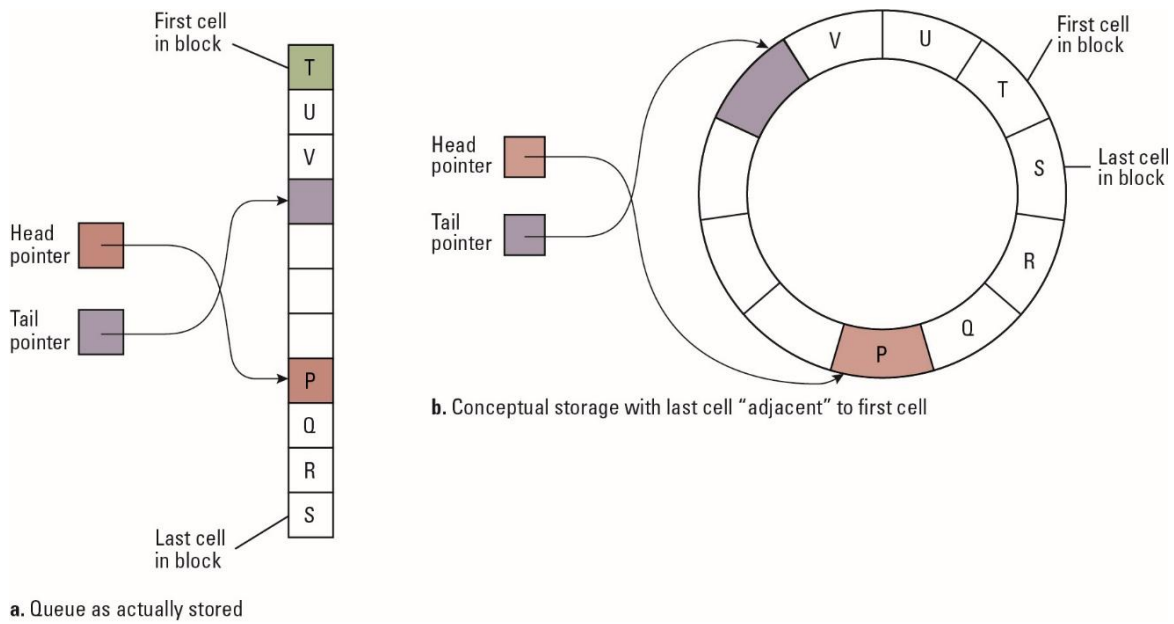
實作 Queue

- Queue 儲存在連續記憶體
- Head Pointer 指向 Queue 的 Head，Tail Pointer 指向 Queue 的 Tail

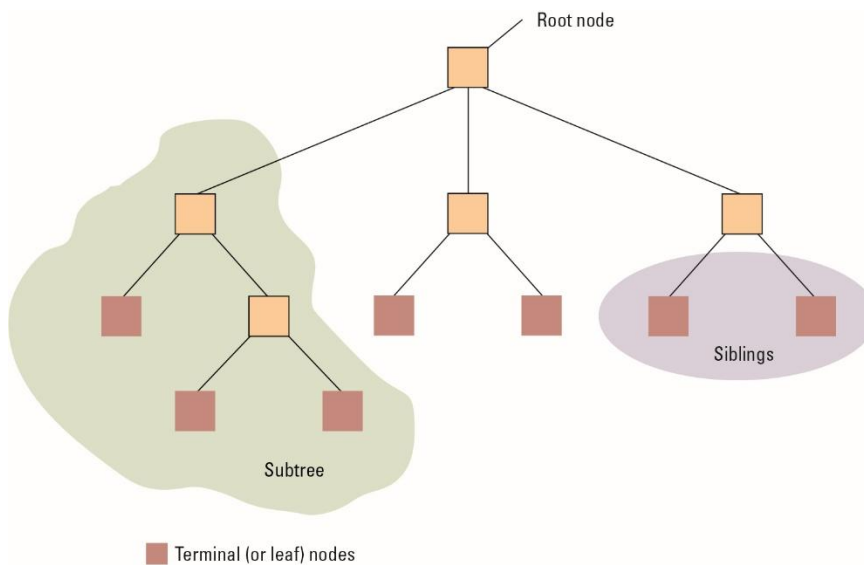


- Queue 通常會實作成 Circular Queue，避免 Queue 一直向右移

範圍尾端的 Pointer 就指向最前端



Tree 樹



- **Node** 節點：tree 裡面的一項
- **Root Node** 根節點：最上層的節點
- **Terminal** 終端 / **Leaf Node**：最底下的節點
- **Parent** 父節點：直接連接到某節點上一層的節點
- **Child** 子節點：直接連接到某節點下一層的節點
- **Ancestor**：所有 Parent 以及 Parent 的 Parent
- **Descendent**：所有 Child 以及 Child 的 Child
- **Siblings**：同屬同一個 Parent 的節點們
- **Subtree**：把某個 node 當成 root 所形成的 tree，形成 parent 的一個 **Branch**
- **Depth** 深度：某節點到 Root Node 的距離（層數）
- **Maximum Depth** 最大深度：從 Leaf Node 到 Root Node 的最長距離

// 課本把 Depth 直接解釋成 Maximum Depth

☆ Binary Tree 二元樹：每個節點最多有兩個 Child 的 Tree Left Child、Right Child

實作 Binary Tree

- 每個 node 都有兩個 Child，儲存成連續的 Array

$A[1]$ = root node

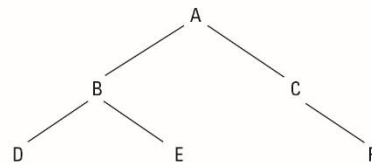
$A[2], A[3]$ = children of $A[1]$

$A[4], A[5], A[6], A[7]$ = children of $A[2]$ and $A[3]$

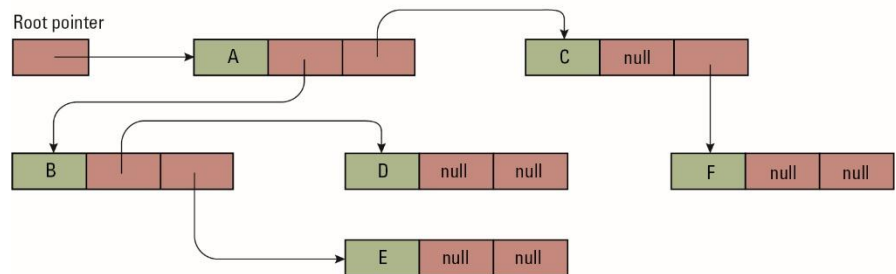
- 每個 Node 的 structure 包含：

- Data
- Left Child Pointer
- Right Child Pointer

Conceptual tree

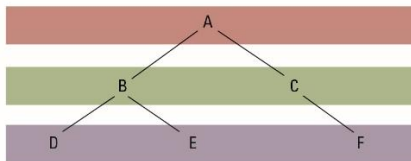


Actual storage organization

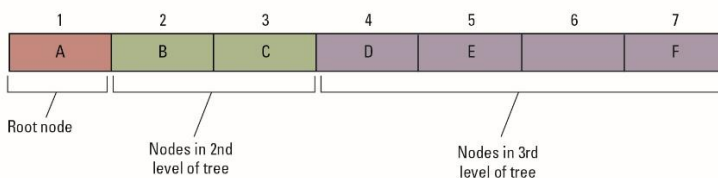


不用 Pointer 的做法

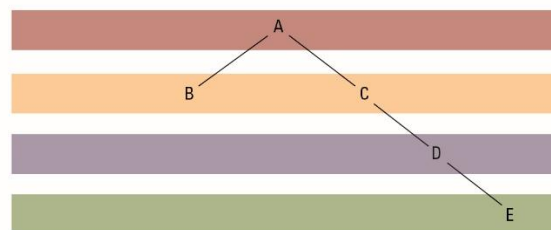
Conceptual tree



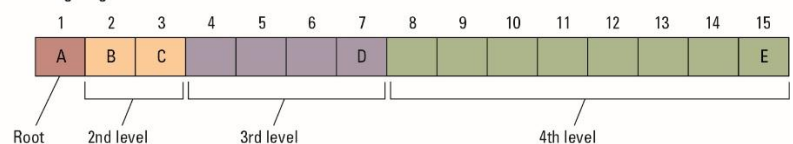
Actual storage organization



Conceptual tree



Actual storage organization



Static vs. Dynamic Structure

- Static 靜態：結構的形狀與大小不會隨操作變化
- Dynamic 動態：結構會隨操作而擴展或縮小

Pointer 指標

儲存「某項資料儲存的記憶體位置」的空間

CPU 中的 Program Counter 有時稱作 Instruction Pointer

Memory Leak

程式在動態配置記憶體後，**沒有在不再需要時釋放它。**

導致這塊記憶體無法再被程式使用，但又依然佔著系統資源，讓可用記憶體空間越來越少。

User-Defined Data Types

- 用 Aggregate Structure 定義新的 Type

```
struct EmployeeType
{
    char    Name[25];
    int     Age;
    real    SkillRating;
}
```

- 用新的 Type 宣告變數

```
struct EmployeeType DistManager, SalesRep1;
```

- 可以同時包含 data 和 function (**Abstract Data Type** , ADT)

支援 ADT 的語言通常具備兩項功能：

- ◆ 能把資料與對資料的操作定義在同一單位中 (例如 class)
- ◆ 能把 ADT 內部結構隱藏起來的機制 (例如 OOP 的封裝)

```
interface StackType
{
    public int pop();
    public int push(int item);
    public boolean isEmpty();
    public boolean isFull();
}
```

Type v.s. Instance

類似 OOP 中 Class 和 Object 的關係 做餅乾的模具本身不是餅乾

- Type 是型態的**模板**，定義資料型態的結構
- Instance 是實際在程式中操作的**實體**

機器語言中的 Pointer

- **Immediate addressing 立即**：指令本身就含有資料

```
MOV R1 #5
```

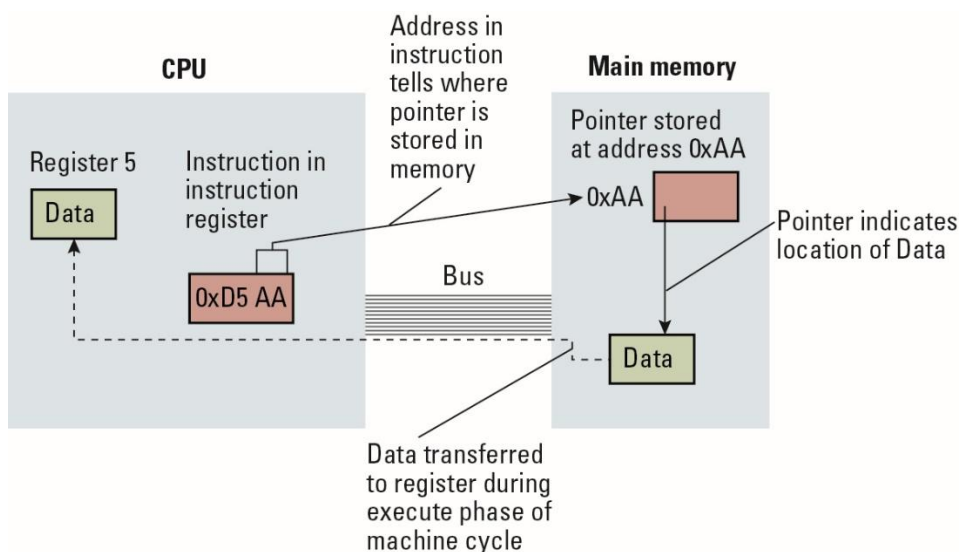
這個 5 就是資料本身，把 5 這個值**直接**放入 **R1 暫存器**

沒有 Pointer

- **Direct addressing 直接**：指令裡放資料所在的**記憶體地址**

```
MOV R1 [1001]
```

[1001] 代表**記憶體地址 1001** 所放的資料，把**記憶體地址 1001** 裡的內容**取出**放入 **R1 暫存器**



- **Indirect addressing 間接**：指令裡放「存放資料所在地址」的位置

`MOV R1 [R2]` 假設 R2 裡面存放 1003

[R2] 代表到 R2 暫存器中存放的「1003」這個地址去取值，把 1003 的內容取出放入 R1 暫存器

